STUDENT RECORDS AND ACADEMIC PLANNING

How the project uses its live data workflows

This guide explains why the admin Data Structures page exists and how it uses the CollegeManagementWeb database. Section headings describe the college task; implementation techniques are explained in the text rather than used as feature headings.

Project purpose	Implementation in the current project
Review and organize student records	Accepted students load from student_applications and students; temporary list and tree views are l
Phase 3: Find and organize students for admission decisions (CLO2)	Partial and exact roll-number lookup plus sorting by roll number, name, or CGPA, with displayed co
Phase 4: Plan teacher support for accepted students (CLO3)	Resource proposals use accepted students and active teachers for advising balance, tutoring-hou
Explore relationships between students	Saved student_connections become a list and matrix view; staff can explore reachable students fro

Data safety: Students are included in these views only when an application has status Accepted. Pending applicants appear only as application-number tokens in the review queue. Search, sorting, trees, and planning use in-memory snapshots and do not change student or application records. Planning results are not saved as assignments.

How student review works in the project

The admin page requires an admin login and reads accepted student roll number, name, department, semester, and CGPA from the database. It separately reads active teachers, pending application numbers, and saved student connections. The list, stack, queue, and tree controls demonstrate possible review workflows without writing to the source records.

What staff need to do	How the page supports it
Adjust a temporary list of accepted students	Insert, unlink, and traverse in-page student nodes; the student table is not deleted or changed.
Review the newest selected item first	A linked-node stack follows last-in, first-out order.
Process pending applications in submission order	A linked-node queue uses application-number tokens ordered by submission time; queue actions
Inspect a structured view of the accepted cohort	The binary tree fills child positions level-by-level; inorder, preorder, and postorder show different r
Locate a student using the roll number	A separate ordered tree compares roll numbers and displays the comparison path for an exact lo

For student relationships, the existing Student Network page stores connections in student_connections. The Data Structures page reads those same saved links, shows them as a neighbor list and a matrix, and explores the connected component from a chosen accepted student using a FIFO traversal queue.

Open Data Structures from the admin sidebar. Use Student Network to create persistent connections; the visual and traversal examples do not modify those links.

Phase 3: Find and organize approved students for admission decisions (CLO2)

Why it is implemented: Administrators need to locate an eligible student and arrange records for admission review without exposing applicants whose admission is still pending.

How partial lookup works: The PHP page selects students with an Accepted application and passes only those records as a read-only snapshot. The browser normalizes the entered query to lower case, checks each student's name and roll number in turn, and collects every partial match. It reports how many records it inspected.

What staff need to do	Implementation in this project
Find an exact roll number	The browser sorts a copy by roll number, checks the middle record, then moves the low or high boundary to
Order records for review	The selected key compares roll number/name as text or CGPA numerically. Insertion ordering shifts larger i
Verify the feature in the app	Enter a partial name or roll number; then enter a complete roll number. Choose roll number, name, or CGP

The lab explains the search and ordering techniques as implementation details, not as the purpose of the feature. For the regular database search workflow, admins can also open search_sort.php from the page link.

Phase 4: Plan teacher support for accepted students (CLO3)

Why it is implemented: Academic teams need a fair starting point for distributing advising, prioritizing limited tutoring time, and checking whether adviser assignments can satisfy workload and semester constraints.

How advising is balanced: The page pairs accepted students with active teachers. For each student it chooses the least-loaded teacher in the same department when possible; if none is available, it chooses the least-loaded active teacher overall. Ties are resolved by teacher name. It initializes an empty list for each teacher, updates that teacher's list after each choice, and prints the resulting proposal. This greedy proposal is displayed only and is not saved.

How tutoring time is allocated: The admin chooses a whole-number budget from 1 to 50 hours. A 0/1 knapsack calculation selects the combination with the largest estimated support need that fits the budget. Estimated hours are derived from CGPA as max(1, min(10, 11 - floor(CGPA))); the need score is max(1, round((10 - CGPA) * 10)). A table indexed by the first i students and each budget stores the best achievable score. Each cell chooses between skipping the student and including them (only if their hours fit); tracing the table backward recovers the chosen students. Each student can be chosen at most once. These are demonstration heuristics, not stored support plans or official risk assessments.

How adviser constraints are checked: The planner tries active teachers in same-department preference order, tracks each teacher's current load and assigned semesters, and backs up to try another teacher when a choice conflicts. It caps each teacher at ceil(accepted student count / active teacher count) and does not assign two students from the same semester to one teacher. For each student it tentatively updates the load, semester set, and assignment; if a later student cannot be placed, it undoes those changes and tries another teacher. If no valid combination exists, it reports that no capacity-safe plan was found. These proposals are not persisted.

To demonstrate CLO3, run the advising plan, change the tutoring-hour budget and run that plan, then run the adviser assignment. Explain that the three results answer different planning needs and are previews only; official teacher assignments still require a separate database-backed workflow.